ReconForge API Reference

> Version 1.0 — Last updated: 2026-03-21

Core Classes

Runner (`core/runner.py`)

Secure subprocess execution engine.

```
class Runner:
    def __init__(self, logger: ReconLogger, timeout: int = 300, dry_run: bool = False)
    def check_tool(self, tool_name: str) -> bool
    def run(self, command: Union[str, Sequence[str]], timeout: Optional[int] = None,
            output_file: Optional[Path] = None, env: Optional[dict] = None,
            stdin_data: Optional[str] = None) -> RunResult
    def run_or_raise(self, command, timeout=None, output_file=None, env=None,
stdin_data=None) -> RunResult
    def check_tool_or_raise(self, tool_name: str) -> bool
    def get_command_log(self) -> List[str]
    def save_command_log(self, path: Path)
```

```
@dataclass
class RunResult:
    command: str
    returncode: int
    stdout: str
    stderr: str
    duration: float
    success: bool
    output_file: Optional[str] = None
```

Helper functions:

- `validate_arg(value: str, label: str = "argument") -> str` — rejects shell metacharacters
- `quote_args(*args: str) -> str` — deprecated, use `list[str]` instead

ConfigLoader (`core/config_loader.py`)

```
class ConfigLoader:
    def __init__(self, config_dir: Optional[Path] = None)
    def load(self, name: str) -> dict
    def get_tool_config(self, tool_name: str) -> dict
    def get_profile(self, profile_name: str) -> dict
    def tool_config(self, tool_name: str) -> ToolConfig
    def get(self, name: str, key: str, default: Any = None) -> Any
```

ToolConfig (`core/tool_config.py`)

```
class ToolConfig:
    def __init__(self, config: Optional[ConfigLoader], tool_name: str)

    # Properties
    binary: str
    alt_binary: str
    required: bool
    default_timeout: int
    description: str
    detection: str
    opt_in_only: bool
    has_config: bool

    # Mode/profile methods
    def mode_timeout(self, mode: str, default: int) -> int
    def mode_args(self, mode: str, default: str = "") -> str
    def mode_detection(self, mode: str, default: str = "medium") -> str
    def mode_value(self, mode: str, key: str, default: Any = None) -> Any
    def mode_requires_root(self, mode: str) -> bool

    # Safety/collection
    def safety(self, key: str, default: Any = None) -> Any
    def collection(self, method: str, key: str, default: Any = None) -> Any

    # Generic
    def get(self, key: str, default: Any = None) -> Any
    def effective_timeout(self, mode: Optional[str], caller_default: int) -> int
```

ProfileLoader (`core/profile_loader.py`)

```
class ProfileLoader:
    def __init__(self, config: ConfigLoader, opsec_mode: str = "normal", module: str =
"")

    # Properties
    profile_data: Dict[str, Any]
    opsec_mode: str
    timing: Dict[str, Any]
    allowed_noise: List[str]
    nmap_timing: str
    scan_delay: str
    max_retries: int

    # Methods
    def section(self, key: str) -> Dict[str, Any]
    def get(self, dotted_key: str, default: Any = None) -> Any
    def is_technique_enabled(self, technique: str) -> bool
    def enabled_phases(self) -> Optional[List[str]]
```

FindingsManager (`core/findings_manager.py`)

```
class FindingsManager:
    VALID_SEVERITIES = {"critical", "high", "medium", "low", "info"}
    VALID_CONFIDENCES = {"confirmed", "high", "medium", "low", "heuristic"}

    def __init__(self, strict: bool = True)
    def add(self, finding_type: str, severity: str, confidence: str,
            target: str, module: str, description: str,
            evidence: str = "", recommendation: str = "",
            phase: str = "", references: Optional[List[str]] = None) -> Finding
    def get_all(self) -> List[Finding]
    def get_by_severity(self, severity: str) -> List[Finding]
    def get_by_confidence(self, confidence: str) -> List[Finding]
    def get_by_module(self, module: str) -> List[Finding]
    def get_heuristic_findings(self) -> List[Finding]
    def count_by_severity(self) -> Dict[str, int]
    def count_by_confidence(self) -> Dict[str, int]
    def to_json(self) -> str
    def to_markdown(self) -> str
    def save_json(self, path: Path)
    def save_markdown(self, path: Path)

    @property
    def clamped_count(self) -> int
```

```
@dataclass
class Finding:
    id: str
    finding_type: str       # vulnerability, misconfiguration, exposure, credential,
attack_vector, information, assessment, prioritisation
    severity: str           # critical, high, medium, low, info
    confidence: str         # confirmed, high, medium, low, heuristic
    target: str
    module: str
    phase: str
    description: str
    evidence: str
    recommendation: str
    references: List[str]
    timestamp: str
```

LootManager (`core/loot_manager.py`)

```
class LootManager:
    def __init__(self, encrypt: bool = False)
    def add(self, loot_type: str, value: str, source: str, module: str,
            confidence: str = "medium", metadata: Optional[Dict] = None) -> LootItem
    def add_credential(self, username: str, password: str, source: str, module: str,
                       service: str = "", confidence: str = "confirmed") -> LootItem
    def add_hash(self, hash_value: str, hash_type: str, source: str, module: str,
                 username: str = "") -> LootItem
    def add_user(self, username: str, source: str, module: str,
                 domain: str = "", confidence: str = "high") -> LootItem
    def add_share(self, share_path: str, permissions: str, source: str, module: str,
                  anonymous: bool = False) -> LootItem
    def add_service(self, service: str, version: str, port: int,
                    source: str, module: str) -> LootItem
    def get_by_type(self, loot_type: str) -> List[LootItem]
    def get_all(self) -> List[LootItem]
    def get_users(self) -> List[str]
    def get_credentials(self) -> List[Dict]
    def to_json(self) -> str
    def save(self, path: Path)
    def summary(self) -> Dict[str, int]

    @staticmethod
    def load_encrypted(path: Path) -> str
```

```
@dataclass
class LootItem:
    loot_type: str      # credential, hash, token, share, user, service
    value: str
    source: str
    module: str
    confidence: str
    metadata: Dict
    timestamp: str
```

CredentialVault (`core/credential_vault.py`)

```
class CredentialVault:
    def __init__(self, encrypt: bool = False, key_path: Optional[Path] = None)

    # Add methods
    def add_password(self, username, password, *, source="", module="", domain="",
service="", confidence="confirmed", metadata=None) -> Optional[Credential]
    def add_hash(self, hash_value, hash_type, *, username="", source="", module="",
domain="", service="", confidence="confirmed", metadata=None) -> Optional[Credential]
    def add_token(self, token, token_type="bearer", *, username="", source="", mod-
ule="", service="", confidence="confirmed", metadata=None) -> Optional[Credential]
    def add_api_key(self, key, *, username="", source="", module="", service="", con-
fidence="confirmed", metadata=None) -> Optional[Credential]
    def add_ssh_key(self, key_material, *, username="", source="", module="", ser-
vice="ssh", confidence="confirmed", metadata=None) -> Optional[Credential]
    def add_username(self, username, *, source="", module="", domain="", confidence="h
igh", metadata=None) -> Optional[Credential]

    # Query methods
    def get_all(self) -> List[Credential]
    def get_by_type(self, cred_type: str) -> List[Credential]
    def get_passwords(self) -> List[Credential]
    def get_hashes(self) -> List[Credential]
    def get_tokens(self) -> List[Credential]
    def get_usernames(self) -> List[str]
    def get_for_service(self, service: str) -> List[Credential]
    def get_for_module(self, module: str) -> List[Credential]
    def get_validated(self) -> List[Credential]
    def count(self) -> int
    def summary(self) -> Dict[str, int]

    # Integration
    def ingest_from_loot(self, loot_manager) -> int
    def contribute_to_loot(self, loot_manager) -> int
    def mark_validated(self, cred_id: str, validated: bool = True)

    # Persistence
    def to_json(self) -> str
    def save(self, path: Path)
    def load(self, path: Path)

    # Export
    def export_usernames(self, path: Optional[Path] = None) -> List[str]
    def export_passwords(self, path: Optional[Path] = None) -> List[str]
    def export_hashes(self, path: Optional[Path] = None) -> List[str]
```

```
@dataclass
class Credential:
    id: str
    cred_type: str      # password, hash_ntlm, hash_ntlmv2, hash_other, token_jwt,
token_bearer, api_key, ssh_key, username, cookie, certificate
    username: str
    secret: str
    domain: str
    service: str
    source: str
    module: str
    confidence: str
    validated: bool
    metadata: Dict[str, Any]
    timestamp: str
```

EngagementManager (`core/engagement.py`)

```
class EngagementManager:
    def __init__(self, name="", client="", operator="", scope=None, tags=None, notes="
")

    # Lifecycle
    status: str  # planning, active, paused, completed, cancelled
    def start(self)
    def pause(self)
    def resume(self)
    def complete(self)
    def cancel(self)

    # Timeline
    def record_action(self, module: str, action: str, detail: str = "", operator: str
= "")
    def record_module_result(self, module: str, result: Dict[str, Any])
    def get_timeline(self) -> List[Dict[str, str]]

    # Aggregation
    def update_findings_summary(self, findings_manager)
    def update_loot_summary(self, loot_manager)
    findings_summary: Dict[str, int]
    loot_summary: Dict[str, int]
    modules_run: List[str]

    # Persistence
    def to_dict(self) -> Dict[str, Any]
    def to_json(self) -> str
    def save(self, path: Path)
    def to_markdown(self) -> str

    @classmethod
    def load(cls, path: Path) -> "EngagementManager"
```

AttackWorkflow (`core/attack_workflow.py`)

```
class AttackWorkflow:
    steps: List[WorkflowStep]
    attack_paths: List[AttackPath]
    current_phase: str
    rabbit_holes: List[str]

    def add_step(self, phase, hypothesis, command, justification, alternatives=None) -
> WorkflowStep
    def record_result(self, result: str)
    def add_attack_path(self, name, description, steps, risk, prerequisites=None, ref-
erences=None) -> AttackPath
    def suggest_next(self, command: str, justification: str, priority: str = "medium")
    def get_suggestions(self) -> List[Dict]
    def add_rabbit_hole(self, description: str)
    def to_markdown(self) -> str
```

WorkflowOrchestrator (`core/workflow_orchestrator.py`)

```
class WorkflowOrchestrator:
    def __init__(self, targets=None, opsec_mode="normal", output_base="outputs",
                 verbose=False, dry_run=False, timeout=600, encrypt_loot=False,
                 credential_vault=None, engagement=None)

    def add_step(self, module_name, *, condition=None, config=None, critical=False,
description="") -> "WorkflowOrchestrator"
    def add_full_recon(self) -> "WorkflowOrchestrator"
    def clear_steps(self)
    def run(self) -> Dict[str, Any]

    context: WorkflowContext
    results: List[StepResult]

    @classmethod
    def full_recon(cls, targets: List[str], **kwargs) -> "WorkflowOrchestrator"
    @classmethod
    def targeted(cls, targets: List[str], modules: List[str], **kwargs) -> "Workflo-
wOrchestrator"
```

```
class WorkflowContext:
    targets: List[str]
    live_hosts: List[str]
    open_ports: Dict[str, List[int]]
    services: Dict[str, Set[str]]
    domains: List[str]
    urls: List[str]

    def has_service(self, service: str) -> bool
    def has_port(self, port: int) -> bool
    def has_domain(self) -> bool
    def has_url(self) -> bool
    def host_count(self) -> int
    def add_hosts(self, hosts: List[str])
    def add_ports(self, host: str, ports: List[int])
    def add_services(self, host: str, services: List[str])
    def add_domain(self, domain: str)
    def add_url(self, url: str)
    def store_result(self, module: str, result: Dict[str, Any])
    def to_dict(self) -> Dict[str, Any]
```

NotesManager (`core/notes_manager.py`)

```
class NotesManager:
    def __init__(self, target: str = "")
    def add(self, note: str, category: str = "general")
    def add_phase_start(self, phase: str)
    def add_phase_end(self, phase: str, summary: str = "")
    def add_finding_note(self, description: str)
    def add_command_note(self, command: str, result_summary: str = "")
    def to_markdown(self) -> str
    def save(self, path: Path)
```

OutputManager (`core/output_manager.py`)

```
class OutputManager:
    def __init__(self, base_dir: str = "outputs", target: str = "default")

    def module_dir(self, module: str) -> Path
    def raw_dir(self, module: str) -> Path
    def parsed_dir(self, module: str) -> Path
    def findings_file(self, module: str, ext: str = "json") -> Path
    def session_file(self, module: str) -> Path
    def commands_log(self, module: str) -> Path
    def attack_paths_file(self, module: str) -> Path
    def report_file(self, module: str) -> Path
    def loot_file(self, module: str) -> Path
    def generate_engagement_report(self, findings, loot, workflow,
                                   modules_run=None, opsec_mode="normal", html=False)
-> Path
```

Validators (`core/validators.py`)

```
def validate_ip(value: str) -> str
def validate_cidr(value: str) -> str
def validate_ip_or_cidr(value: str) -> str
def validate_hostname(value: str) -> str
def validate_target(value: str) -> str        # IP, CIDR, hostname, or URL
def validate_port(value) -> int
def validate_port_range(value: str) -> str    # nmap-style: "22,80-443,8080" or "-"
def parse_port_list(value: str) -> List[int]
def validate_url(value: str) -> str
def validate_domain(value: str) -> str
```

All raise typed exceptions from `core.exceptions`:
- `TargetValidationError` (for IPs, CIDRs, hostnames)
- `PortValidationError` (for ports)
- `ValidationError` (for URLs, domains)

Logger (`core/logger.py`)

```
class ReconLogger:
    def __init__(self, name: str = "reconforge", log_dir: Optional[Path] = None, verb-
ose: bool = False)
    def debug(self, msg: str)
    def info(self, msg: str)
    def warning(self, msg: str)
    def error(self, msg: str)
    def critical(self, msg: str)
    def command(self, cmd: str)             # Logs with credential sanitization
    def finding(self, severity: str, description: str)
    def loot(self, loot_type: str, value: str)
    def credential(self, username: str, source: str)
    def phase_start(self, phase: str)
    def phase_end(self, phase: str, summary: str = "")
    def workflow(self, step: str, detail: str = "")
    def with_context(self, **fields) -> ContextLogger

def sanitize_log(message: str) -> str       # Redacts passwords, hashes, tokens, API
keys
```

OpsecChecker (`core/opsec_checks.py`)

```
class OpsecChecker:
    def __init__(self, mode: str = "normal", logger=None)
    def check(self, technique: str) -> bool   # Returns True if allowed
    def warn(self, technique: str) -> Optional[str]  # Returns warning string if risky
    def set_mode(self, mode: str)
```

Exceptions (`core/exceptions.py`)

```
class ReconForgeError(Exception)              # Root exception
class ConfigError(ReconForgeError)            # Config file errors
class ProfileNotFoundError(ConfigError)       # Missing OPSEC profile
class ValidationError(ReconForgeError)        # Input validation failures
class TargetValidationError(ValidationError)  # Invalid target
class PortValidationError(ValidationError)    # Invalid port
class ExecutionError(ReconForgeError)         # Subprocess failures
class ToolNotFoundError(ExecutionError)       # Tool not on PATH
class TimeoutError(ExecutionError)            # Command timeout
class ModuleError(ReconForgeError)            # Module-level errors
class PhaseError(ModuleError)                 # Phase-level errors
class WorkflowError(ReconForgeError)          # Workflow errors
class WorkflowAbortedError(WorkflowError)     # Deliberate workflow abort
class CredentialVaultError(ReconForgeError)   # Vault operation failures
class EngagementError(ReconForgeError)        # Engagement lifecycle errors
class EngagementNotFoundError(EngagementError)  # Missing engagement file
```

Module Classes

NetworkModule (`modules/network/network_module.py`)

```
class NetworkModule:
    MODULE_NAME = "network"
    VALID_PHASES = ["discovery", "scanning", "enumeration", "authentication"]

    def __init__(self, target, output_base="outputs", opsec_mode="normal",
                 verbose=False, dry_run=False, timeout=600, config_dir=None, en-
crypt_loot=False)
    def run(self, phases=None, brute_force=False) -> Dict
```

WebModule (`modules/web/web_module.py`)

```
class WebModule:
    def __init__(self, target, output_base="outputs", opsec_mode="normal",
                 verbose=False, dry_run=False, timeout=600, encrypt_loot=False)
    def run(self, phases=None, opt_in=False) -> Dict
```

APIModule (`modules/api/api_module.py`)

```
class APIModule:
    def __init__(self, target, output_base="outputs", opsec_mode="normal",
                 verbose=False, dry_run=False, timeout=600, encrypt_loot=False,
                 headers=None, auth_token=None)
    def run(self, phases=None, opt_in=False) -> Dict
```

SurfaceModule (`modules/surface/surface_module.py`)

```
class SurfaceModule:
    def __init__(self, target, output_base="outputs", opsec_mode="normal",
                 verbose=False, dry_run=False, timeout=600, config_dir=None,
                 encrypt_loot=False)
    def run(self, phases=None) -> Dict
```

> **Note:** `encrypt_loot` is accepted by the constructor but is **not exposed as a CLI flag** on the `surface` subcommand. The workflow orchestrator can pass it programmatically.

ADModule (`modules/ad/ad_module.py`)

```
class ADModule:
    def __init__(self, target, domain="", output_base="outputs", opsec_mode="normal",
                 verbose=False, dry_run=False, timeout=600, username="", password="",
                 dc_ip="", encrypt_loot=False)
    def run(self, phases=None) -> Dict
```

Phase Base Classes

All share identical constructor signatures:

```
class <Module>PhaseBase(ABC):
    PHASE_NUMBER: int
    PHASE_NAME: str
    PHASE_DESCRIPTION: str

    def __init__(self, logger, runner, config, output_dir, findings, loot,
                 workflow, notes, opsec, opsec_mode="normal", profile=None)

    @abstractmethod
    def run(self, **kwargs) -> Dict[str, Any]: ...
```

Implementations: `NetworkPhaseBase`, `WebPhaseBase`, `APIPhaseBase`, `SurfacePhaseBase`, `ADPhaseBase`

Surface Intelligence Classes

CorrelationEngine (`modules/surface/intelligence/correlation_engine.py`)

```
@dataclass
class CorrelatedService:
    canonical_name: str
    ports: List[int]
    versions: List[str]
    urls: List[str]
    detection_methods: Set[str]
    high_value: bool
    confidence: float
    # ... (see source)

@dataclass
class AttackSurfaceMap:
    target: str
    services: Dict[str, CorrelatedService]
    by_category: Dict[str, List[str]]
    total_ports: int
    total_services: int
    high_value_count: int
```

ConfidenceScorer (`modules/surface/intelligence/confidence_scorer.py`)

```
class ConfidenceScorer:
    WEIGHTS = {
        "port_match": 0.25,
        "banner_match": 0.25,
        "version_detected": 0.20,
        "multi_detection": 0.20,
        "http_confirmed": 0.10,
    }
    LABELS = [(0.80, "confirmed"), (0.60, "high"), (0.40, "medium"), (0.0, "low")]

@dataclass
class ConfidenceResult:
    score: float        # 0.0 - 1.0
    label: str          # confirmed, high, medium, low
    signals: Dict[str, bool]
    explanation: str
```

ServiceDeduplicator (`modules/surface/intelligence/deduplicator.py`)

```
class ServiceDeduplicator:
    def deduplicate_ports(self, ports: List[Dict], services: List[Dict]) -> List[Dict]
```

API reference validated: 2026-03-21 — 348/348 tests passing